Atividade Avaliativa A2 - Parte 2

Relatório Final: Sistema de Agendamento de Monitorias

João Francisco Barcala Paulo Lorenzo Brugnolo Rosa

22 de Outubro de 2026

Este documento tem três seções: (1) o projeto inicial; (2) o relatório de execução, que também serve como README; (3) a documentação técnica.

Seção 1 - Projeto Inicial

Conteúdo do arquivo `projeto_inicial.md`, com os testes citados pelos nomes que existem em `testes/`. As mudanças estão na Seção 2.

Projeto Inicial: Reserva de Horários

História de Usuário Escolhida (Prioridade Alta): Como aluno, eu gostaria de poder marcar os horários e matérias das monitorias que eu queira participar.

1. Descrição das classes/funções

Para implementar esta história, o sistema utilizará as seguintes classes principais:

- **`Aluno` (herda de `Usuario`):** Representa o usuário autenticado que deseja realizar a reserva. É responsável por armazenar suas próprias informações de perfil e manter o histórico ou contador de reservas feitas na semana vigente, garantindo que a validação de limites por usuário seja possível.
- **`Monitoria`:** Entidade que armazena os dados do atendimento oferecido (disciplina, horário, monitor responsável, capacidade total e vagas disponíveis). Sua responsabilidade é gerenciar seu próprio estado de ocupação, permitindo ou negando a entrada de novos alunos com base na capacidade máxima (individual ou em grupo).
- **`ControladorReserva`:** Classe orquestradora (ou módulo de funções) responsável pela regra de negócio principal. Ela recebe a intenção de reserva do `Aluno` para uma determinada `Monitoria`, faz as validações necessárias (checa autenticação, verifica se há vagas e se o aluno não estourou o limite semanal) e, se tudo estiver correto, efetiva a reserva deduzindo uma vaga da sessão.

2. Mapeamento de testes

Os critérios de aceitação da história acima serão verificados através do framework `unittest`, garantindo cenários de sucesso e exceção:

- **Critério de Aceitação 1:** O aluno só pode reservar uma monitoria se houver vagas disponíveis.
  Teste correspondente: `test_reserva_com_vaga_entra_na_agenda` (Cenário de Sucesso)
- **Critério de Aceitação 2:** O sistema deve impedir a reserva caso a capacidade máxima (individual ou grupo) da monitoria já tenha sido atingida.
  Teste correspondente: `test_reserva_lotada_nao_entra_na_agenda_do_aluno` (Cenário de Falha/Exceção)
- **Critério de Aceitação 3:** O aluno não pode exceder o limite de reservas semanais estipulado pela coordenação (ex: 2 reservas por disciplina).
  Teste correspondente: `test_terceira_reserva_na_mesma_disciplina_bloqueada` (Cenário de Falha)
- **Critério de Aceitação 4:** Apenas usuários com a classe/perfil de aluno autenticado podem realizar a ação de reserva.
  Teste correspondente: `test_usuario_inicia_role_definida` e `test_login_valido_retorna_usuario_com_papel_aluno` (Falha/Permissão)

Seção 2 - Relatório de Execução (README)

Como executar. Python: 3.10 ou superior (testado no 3.13). **Bibliotecas:** nenhuma externa, só a biblioteca padrão (`unittest`, `datetime`, `threading`, `abc`, `typing`, `getpass`). **Arquivo principal:** `main.py`, na raiz do projeto (código em `codigo/`, testes em `testes/`). Todos os comandos devem ser rodados na raiz:

- Sistema: `python main.py`. Usuários de exemplo: `aluno`, `monitor` e `professor`, todos com senha `1234`.
- Testes: `python -m unittest discover testes -v` (62 testes, todos passando).

Plano x realizado.

Data	Etapa planejada	Resp. no plano	O que foi feito, quando e por quem
16/10	Documentação inicial e cronograma (PT1)	João	Feito em dia, por João.
17/10	Criação do GitHub e milestones	Lorenzo	Passou para 18/10, por escolha da equipe (o dia de descanso era 18 e foi trocado para 17). Feito por João.
17/10	Histórias de usuário	João	Passou para 18/10, pelo mesmo motivo. Feito por Lorenzo.
19/10	Milestone 0: Sistema de login	Lorenzo	Feito em dia, por Lorenzo.
19/10	Milestone 1: Classes e suas permissões	João	**Transbordou** para 20 e 21/10, João. Ficou 50% (2 de 4 issues).
20/10	Milestone 2: Integração do login e classes	Lorenzo e João	Feito em dia, só por Lorenzo.
21/10	Milestone 3: Sistema de marcação de horário	João e Lorenzo	Feita em 21/10, finalizada e commitada em 22/10, por João e Lorenzo.
22/10	Revisão geral	Lorenzo e João	Feito em dia, por João e Lorenzo, junto com este relatório.

Atrasos e mudanças no cronograma.

- **Milestone 1 transbordou para 20 e 21/10.** Foi mais complexa do que esperávamos e acabou sendo a milestone mais importante do projeto. Nela montamos o Role-Object (`Usuario` com os papéis `Aluno`, `Monitor` e `Professor`) e as classes de monitoria; a partir dela, com poucas linhas, completamos toda a Milestone 3.
- **Milestone 3 passou para 21/10 e foi finalizada e commitada em 22/10**, por causa do transbordo da Milestone 1, da qual ela dependia.
- **Milestone 2 ficou só com o Lorenzo.** Ela foi feita em dia, mas o João continuou responsável pela Milestone 1, que ainda não tinha terminado.

Issues que não foram concluídas (por isso a Milestone 1 ficou em 50%):

- **Criação e Configuração de Monitorias (Acesso: Monitor)**, nos itens *criar_monitoria() e editar_regras() só para Monitor* e *monitor não edita a monitoria de outro*. A criação foi feita e testada, mas a lógica de edição ficou mais complexa do que o esperado: ao editar, os atendimentos são recriados, e queríamos uma forma leve de guardar a monitoria antes e depois da edição. Com o tempo limitado, o `editar_monitoria` ficou numa versão básica, sem testes.
- **Gestão (Acesso: Professor)**, nos itens *visualizar_relatorio_monitor(monitor)* e o teste do professor lendo esses dados. Por falta de tempo não fizemos a geração de relatórios. Pensando no futuro, deixamos lugares específicos para guardar as monitorias criadas e canceladas (`monitorias`, `monitorias_canceladas` e `alunos_cancelados`), que serviriam para montar o relatório e contar as horas trabalhadas.

Mudanças em relação ao projeto inicial.

- `ControladorReserva` **não foi criado.** Com o Role-Object, só quem tem o papel `Aluno` tem o método de reserva, então a validação ficou em `Aluno.marcar_monitoria` (cancelado, duplicado, limite) e o controle de vagas em `Atendimento.marcar`.
- **Nova classe** `Atendimento`**.** O e-mail diz que cada atendimento dura 30 minutos, então a `Monitoria` é dividida em atendimentos de 30 min, cada um com suas vagas. O `marcar` usa um `Lock` para que dois alunos não peguem a mesma vaga ao mesmo tempo.
- **Nomes dos testes mudaram** em relação aos previstos no projeto inicial; as Seções 1 e 3 citam os testes que existem no código.

Método de trabalho. Usamos TDD: primeiro escrevíamos os testes (fase red) e depois o código até passar (fase green). Cada módulo foi feito para funcionar separado nos seus testes, e depois fizemos os testes que integram as partes (`teste_login_integrado.py`, `teste_integracao_monitoria.py` e `teste_integracao_permissoes.py`).

Seção 3 - Documentação Técnica

1. Histórias de Usuário

História 1 *(implementada)*
Como aluno, **eu quero** marcar os horários e as matérias das monitorias que quero participar, **para** ser atendido sem precisar combinar por mensagem com o monitor.

- **Prioridade:** Alta. É o núcleo do pedido da coordenação, e o próprio e-mail diz que só a reserva funcionando já resolve boa parte do problema.
- **Trecho do e-mail:** "A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento." e "Se a reserva de horários funcionar, já ajuda muito."
- **Critérios de aceitação:**
  1. O aluno pode escolher os horários e as matérias desejadas.
  2. O sistema só pode mostrar os horários disponíveis.
  3. O sistema não pode deixar que 2 alunos agendem o mesmo horário ao mesmo tempo.
  4. O sistema exige que o perfil seja de um aluno para permitir o agendamento.

História 2
Como monitor, **eu quero** registrar os horários em que posso atender, **para** que os alunos só vejam horários em que eu estou realmente disponível.

- **Prioridade:** Alta. Sem os horários cadastrados pelos monitores não existe o que reservar, então a História 1 depende desta.
- **Trecho do e-mail:** "No início do semestre, cada monitor informa os horários em que pode atender".
- **Critérios de aceitação:**
  1. O monitor deve poder registrar e mudar os horários dinamicamente.
  2. O monitor pode definir se o atendimento será individual ou em grupo.
  3. O monitor deve conseguir ver os horários que foram reservados pelos alunos.

História 3
Como monitor, **eu quero** cancelar os horários de atendimento caso algum imprevisto aconteça, **para** que os alunos não descubram o cancelamento só quando chegam na sala.

- **Prioridade:** Média. É um problema real citado no e-mail, mas a reserva já funciona sem ele; por isso vem depois das Histórias 1 e 2.
- **Trecho do e-mail:** "mas às vezes precisa cancelar por causa de prova. Os alunos reclamam que só descobrem o cancelamento quando chegam na sala."
- **Critérios de aceitação:**
  1. O monitor deve ter a opção de cancelar um horário caso um imprevisto aconteça.
  2. O aluno com horário reservado deve ser notificado caso o horário seja cancelado pelo monitor.
  3. O sistema deve cancelar aquele horário e impedir novas reservas nele.

História 4
Como coordenador, **eu quero** que o sistema gere relatórios mensais das horas atendidas pelos monitores, **para** pagar o valor certo das bolsas.

- **Prioridade:** Média. É necessário para o pagamento da bolsa, mas não impede os alunos de usarem o sistema de reservas.
- **Trecho do e-mail:** "A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa não podem ficar visíveis para todo mundo."
- **Critérios de aceitação:**
  1. O sistema deve registrar e somar as horas totais de atendimento.
  2. O relatório só pode ser visto pelo professor ou coordenador.
  3. O relatório deve estar estruturado e padronizado.

História 5
Como professor, **eu quero** ter acesso às dúvidas mais frequentes dos alunos nas monitorias, **para** reforçar esses conteúdos em aula.

- **Prioridade:** Baixa. O e-mail coloca isso como um desejo dos professores e não como um problema que impede o uso do sistema.
- **Trecho do e-mail:** "Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."
- **Critérios de aceitação:**
  1. O sistema deve permitir que o monitor registre as dúvidas dos alunos abordadas na monitoria atendida.
  2. O professor deve ter acesso a um relatório de dúvidas e ver as mais frequentes.
  3. O relatório deve estar estruturado e organizado.

2. Testes de Aceitação

História implementada: **História 1**. Os testes ficam na pasta `testes/` e rodam com `python -m unittest discover testes -v`. Na coluna *Cenário*, S = sucesso e F = falha ou exceção.

Critério	Caso de teste	Arquivo	Cenário
1. Escolher horários e matérias	`test_reserva_com_vaga_entra_na_agenda`	`teste_reservas.py`	S
	`teste_aluno_marcar_monitoria`	`testes_monitorias.py`	S
	`test_reserva_cancelada_nao_entra_na_agenda`	`teste_reservas.py`	F
	`test_bloqueia_reserva_duplicada`	`teste_reservas.py`	F
2. Mostrar só horários disponíveis	`teste_aluno_ver_monitorias`	`testes_monitorias.py`	S
	`test_lista_atendimentos_disponiveis` (atendimento lotado e cancelado não aparecem)	`teste_consulta_monitorias.py`	S/F
	`test_monitoria_cancelada_nao_aparece_na_consulta`	`teste_consulta_monitorias.py`	F
3. Dois alunos não agendam o mesmo horário	`test_reserva_lotada_nao_entra_na_agenda_do_aluno`	`teste_reservas.py`	F
	`teste_segundo_aluno_nao_reserva_atendimento_lotado`	`testes_monitorias.py`	F
	`test_cancelamento_libera_vaga_para_outro_aluno`	`teste_cancelamento_reserva.py`	S
4. Exige perfil de aluno	`test_login_valido_retorna_usuario_com_papel_aluno`	`teste_login_integrado.py`	S
	`test_usuario_inicia_role_definida` (professor não nasce com papel Aluno nem Monitor)	`testes_classes_role_object.py`	F
	`test_login_professor_preserva_perfil`	`teste_login_integrado.py`	F
Regras do projeto inicial: limite por disciplina e duração de 30 min	`test_terceira_reserva_na_mesma_disciplina_bloqueada`	`teste_limite_semanal.py`	F
	`test_intervalo_de_uma_hora_gera_dois_atendimentos`	`teste_duracao_monitoria.py`	S
	`test_rejeita_intervalo_maior_que_30`	`teste_duracao_monitoria.py`	F
Fluxo completo (integração)	`test_login_criacao_e_busca_de_monitoria`	`teste_integracao_monitoria.py`	S

Critério	Caso de teste	Arquivo	Cenário
	`test_promocao_persistida_apos_novo_login`	`teste_integracao_permissoes.py`	S

O que não foi coberto, e por quê:

- **Concorrência real (critério 3).** O `Atendimento.marcar` usa um `Lock`, mas os testes de lotação fazem as reservas em sequência; não há teste com duas threads reservando ao mesmo tempo.
- **Limite "semanal".** O limite de 2 reservas é contado por disciplina entre as reservas ativas, e não por semana do calendário, porque o modelo guarda só o dia da semana e o e-mail ainda não definiu essa regra.
- **Menu do `main.py`.** Não tem testes automáticos porque depende de `input()`; toda a lógica que ele chama já é testada direto nas classes.
- **Edição de monitoria e relatório do professor** (`editar_monitoria`, `contar_horas_trabalhadas`): ficaram sem testes porque fazem parte das issues não concluídas, descritas na Seção 2.